

Individual Report

Domain Model & Repositories (Data Layer)

Name: MA SOVITCHEA

Student ID: e20221575

Project: Course Enrollment System

Group: I4-GIC – Group C

Course: Software Engineering

Lecturer: Mr. ROEUN Pacharoth (TP)

Academic Year: 2025–2026

1. Introduction

In this project, I was responsible for designing and implementing the **Domain Model and Repository Layer**, also referred to as the **Data Layer**. This layer defines the structure of the system by modeling real-world academic entities such as users, courses, schedules, enrollments, and attendance.

The correctness and stability of the data layer are critical, as all other layers—including services, controllers, and the user interface—depend on it.

2. Scope of Responsibility

My responsibilities were limited to the following components:

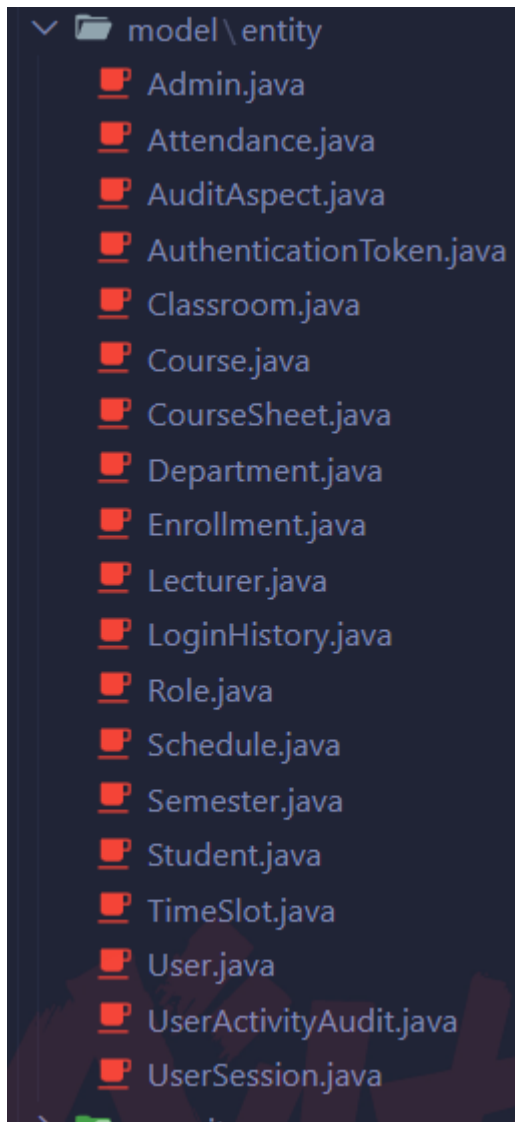
- `model/entity/`
- `repository/`
- `enums/`
- `resources/templates/error/`

I did not implement business logic, controllers, security configuration, or UI components.

3. Domain Entity Design

3.1 Entity Package Structure

All domain entities are organized under the `model/entity/` package to ensure clear separation of concerns and maintainability.



3.2 User and Role Inheritance Design

To support role-based access control, I designed a JPA inheritance hierarchy with **User** as the base entity and **Admin**, **Student**, and **Lecturer** as subclasses. This approach avoids data duplication and allows role-specific attributes to be handled cleanly.

```
@Entity
@Table(name = "users")
@Data
@NoArgsConstructor
@AllArgsConstructor
@EntityListeners(AuditingEntityListener.class)
@Inheritance(strategy = InheritanceType.JOINED)
public class User {
```

3.3 Academic Core Entities

I implemented core academic entities that support course management and scheduling, including `Course`, `Classroom`, `Schedule`, `Semester`, `Department`, and `TimeSlot`. These entities are connected using appropriate JPA relationships.

3.4 Enrollment and Attendance Modeling

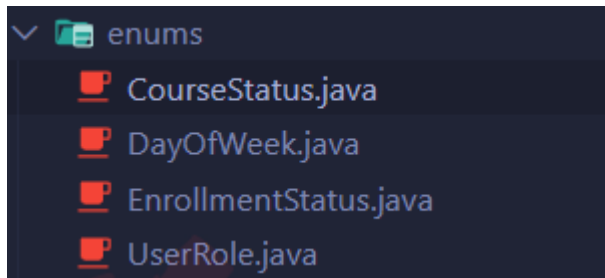
To manage student participation, I designed the `Enrollment` entity to link students with courses and the `Attendance` entity to track attendance per schedule.

3.5 Audit and Session Tracking Entities

For system monitoring and security support, I implemented entities such as `LoginHistory`, `UserSession`, `UserActivityAudit`, and `AuthenticationToken`.

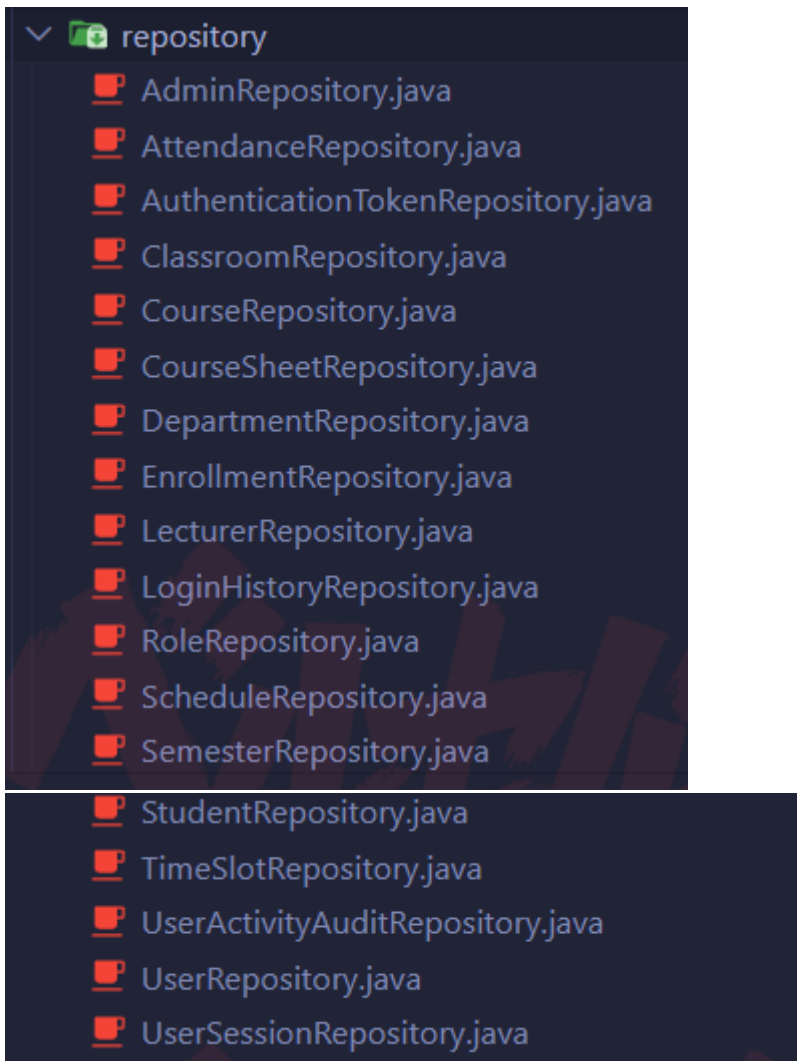
4. Enum Definitions

To enforce data consistency and prevent invalid values, I defined enums under the `enums/` package, including `UserRole`, `CourseStatus`, `EnrollmentStatus`, and `DayOfWeek`.



5. Repository Layer Implementation

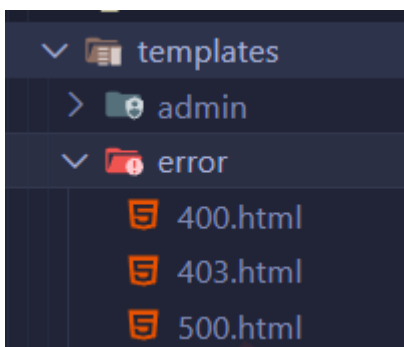
For each entity, I created a corresponding Spring Data JPA repository interface under the `repository/` package. These repositories provide clean abstraction for database operations.



6. Error Template Integration

To improve system robustness and user experience, I added custom error templates under `resources/templates/error/`, including:

- `400.html` – Bad Request
- `403.html` – Forbidden
- `500.html` – Internal Server Error



7. Database Migration Alignment

Although I did not directly write Flyway migration scripts, my entity designs align with the database schema defined in the migration files. This ensures consistency between entity models and the database structure.

8. Challenges and Solutions

Challenge: Designing complex relationships among users, courses, schedules, and enrollments without creating circular dependencies.

Solution: Careful definition of relationship ownership, appropriate use of JPA annotations, and incremental validation of entity mappings.

9. Contribution Summary

My contributions to the project include:

- Complete domain model design
 - JPA inheritance and relationship mapping
 - Enum-based state management
 - Repository abstraction for persistence
 - Error template integration
-